

System Design Specification

Aetheria: Sovereign, Censorship-Resistant Decentralized Publishing Platform

Target Runtime: Freenet P2P Network	Architecture: WASM State Contracts + Delegate	Payment Rail: NWC / Lightning / L2 Micro-rail	Version: 1.0.0-PROD
-------------------------------------	---	---	---------------------

1. Executive Summary & Core Objective

Aetheria is a fully decentralized, serverless publishing and monetization protocol built on top of the **Freenet** key-value peer-to-peer network. Designed to replace centralized platforms like Substack or Medium, Aetheria provides independent journalists, scientific researchers, and content creators with an unstoppable distribution channel that cannot be de-platformed, censored, or restricted by sovereign authorities or private corporate intermediaries.

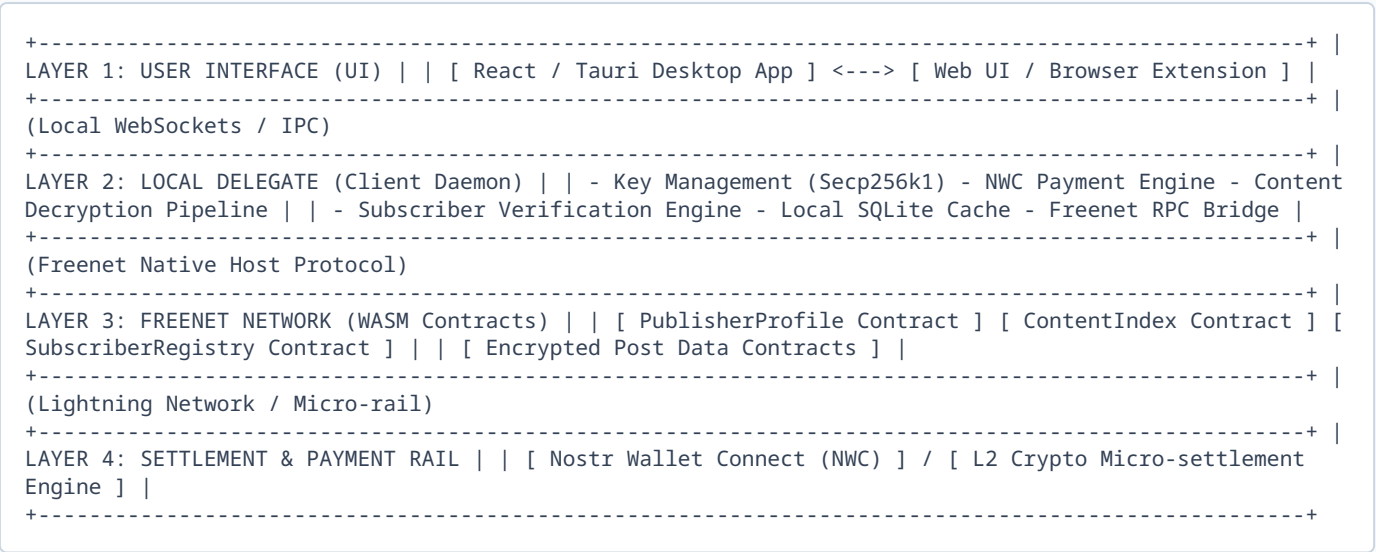
Unlike legacy P2P file systems, Aetheria utilizes Freenet's state-contract architecture featuring WebAssembly (WASM) smart contracts and local node *Delegates* (secure client-side WASM agents). Authors publish encrypted articles directly to the Freenet network, while subscribers maintain direct, non-custodial financial relationships via crypto payment rails (such as Lightning Network / Nostr Wallet Connect). Cryptographic access keys are rotated on epoch cycles and exchanged seamlessly peer-to-peer without central key servers or hosted database dependencies.

Key Architectural Guarantees

- Absolute Censorship Resistance:** Article payloads live in Freenet state contracts distributed across global network peers.
- Zero Platform Intermediaries:** Payments flow directly from subscriber wallet to creator wallet via non-custodial micro-rails.
- Provable Authenticity:** Posts, profile edits, and key rotations are cryptographically signed using creator keypairs (Ed25519/Secp256k1).
- Offline-First & Local Sovereignty:** Readers cache encrypted posts and key schedules locally via their desktop/browser Delegate.

2. System Topology & Layered Architecture

The system is divided into four distinct operational layers, isolating cryptographic execution, state synchronization, payment processing, and user interaction.



Layer	Primary Technology	Core Responsibilities
1. User Interface	React 18 + Tauri / WebAssembly	Renders rich text/markdown editor, subscriber feeds, analytics dashboard, media preview, and local publication settings.
2. Local Delegate	Rust WASM / Native Daemon	Stores identity keys securely, executes automated key derivation, listens to Freenet network state changes, issues Lightning invoices via NWC, and performs local AES-GCM decryption.
3. Freenet Contracts	Rust WASM Freenet SDK	Defines network state, handles CRDT merge operations for subscriber manifests, index entries, and stores chunked encrypted post data.
4. Payment Rail	LN / NWC / L2 Tokens	Executes instant micro-settlements, verifies cryptographic payment receipts (preimages), and signals subscription activation.

3. WASM State Contracts & Data Schemas

Freenet state contracts govern the data stored across the P2P network. Contracts enforce deterministic state update rules using Rust WASM code compiled to WebAssembly targets.

3.1 Publisher Profile Contract (`PublisherProfileContract`)

Maintains public metadata for a creator's publication, including publication name, bio, public identity key, tier definitions, and pointers to the master content index.

```
// Rust Structural Specification for Publisher Profile Contract
pub struct PublisherProfile {
    pub author_pubkey: [u8; 32],           // Ed25519 Public Key
    pub title: String,                     // Publication Title (e.g. "Sovereign Tech Dispatch")
    pub description: String,               // Publication Subtitle/Bio
    pub avatar_freenet_key: Option, // Freenet key pointing to avatar media contract
    pub subscription_tiers: Vec,          // Array of active subscription pricing tiers
    pub content_index_contract_id: String, // Freenet Contract ID for article index
    pub updated_at: u64,                  // Unix Timestamp
    pub signature: [u8; 64],              // Ed25519 Signature over state hash
}

pub struct Tier {
    pub tier_id: u8,
    pub name: String,                     // e.g. "Monthly Supporter", "Annual Patron"
    pub price_sats_per_month: u64,        // Pricing in Satoshis or base token micro-units
    pub features: Vec,
}
```

3.2 Content Index Contract (`ContentIndexContract`)

Acts as a append-only log or table of contents listing all articles published under a publication. It uses a CRDT (Conflict-free Replicated Data Type) structure to merge new posts seamlessly across network nodes.

```
// Rust Structural Specification for Content Index State
pub struct ContentIndexState {
    pub publication_id: [u8; 32],
    pub posts: Vec,
    pub last_sequence_num: u64,
}

pub struct PostMetadataHeader {
    pub post_id: [u8; 16],                // UUID v4 identifier
    pub title: String,                    // Article title (Unencrypted)
```

```

pub summary: String,                // Unencrypted teaser/preview snippet
pub post_contract_id: String,       // Freenet Contract ID containing encrypted payload
pub access_level: AccessTier,       // Public, Tier1, Tier2
pub epoch_id: u32,                  // Epoch required for decryption key derivation
pub published_at: u64,
pub signature: [u8; 64],
}

pub enum AccessTier {
    Public,
    SubscriberOnly { required_tier_id: u8 },
}

```

3.3 Post Data Contract (`PostDataContract`)

Holds the actual markdown content, inline images, and media blobs of an article. Public posts are stored in plaintext; subscriber-only posts are encrypted using symmetric AES-256-GCM prior to network submission.

```

pub struct EncryptedPostPayload {
    pub post_id: [u8; 16],
    pub cipher_text: Vec,           // AES-256-GCM encrypted Markdown payload
    pub nonce: [u8; 12],           // Unique GCM Nonce
    pub auth_tag: [u8; 16],        // Cryptographic Authentication Tag
    pub attachments: Vec, // Embedded media Freenet hashes
}

pub struct MediaAttachment {
    pub attachment_id: String,
    pub mime_type: String,
    pub encrypted_blob_contract_id: String,
}

```

4. Cryptographic Access Control & Key Management

To ensure paid content remains restricted to active subscribers without relying on centralized authorization servers, Aetheria implements an **Epoch-Based Symmetric Key Rotation + ECDH Key Exchange Scheme**.

4.1 Key Architecture Hierarchy

- **Publisher Master Keypair** (SK_{pub}, PK_{pub}): Ed25519 keypair used to sign contract updates and authenticate key distribution.
- **Subscriber Identity Keypair** (SK_{sub}, PK_{sub}): Secp256k1 keypair owned by the reader for asymmetric key exchange.
- **Epoch Key** (K_{epoch}): AES-256 symmetric key generated per billing cycle (e.g. Month 2026-08). Used to encrypt all subscriber-only posts published within that epoch.
- **Encrypted Key Bundle** (E_{sub}): The Epoch Key K_{epoch} encrypted specifically for a subscriber using Elliptic Curve Diffie-Hellman (ECDH).

4.2 Key Derivation & Encryption Mathematical Flow

When a creator publishes a post in Epoch E , the Delegate generates a random 256-bit symmetric key:

$$K_{epoch} = \text{CSPRNG}(256)$$

The post payload M is encrypted using AES-256-GCM:

$$C = \text{AES-GCM-Encrypt}(K_{\text{epoch}}, \text{Nonce}, M)$$

When Subscriber i purchases a subscription, their Delegate sends their public key $PK_{\text{sub},i}$ alongside a payment receipt proof. The Publisher's Delegate calculates the shared ECDH secret:

$$S_i = \text{ECDH}(SK_{\text{pub}}, PK_{\text{sub},i}) = \text{HKDF}(SK_{\text{pub}} \cdot PK_{\text{sub},i})$$

The Publisher's Delegate encrypts the Epoch Key K_{epoch} using the shared secret:

$$E_{\text{key},i} = \text{AES-GCM-Encrypt}(S_i, \text{Nonce}_k, K_{\text{epoch}})$$

This encrypted key bundle $E_{\text{key},i}$ is published to the network's `SubscriberRegistryContract`. Only Subscriber i can decrypt $E_{\text{key},i}$ using their private key $SK_{\text{sub},i}$ to recover K_{epoch} and read the post.

5. End-to-End Operational Workflows

5.1 Workflow A: Article Publication Flow

```
[ Publisher UI ] --( 1. Write Article )--> [ Local Delegate ] | 2. Generate K_epoch (if missing) 3.
Encrypt Payload with AES-GCM | v 4. Sign Contract Updates (Ed25519) | v [ Freenet Network ] <--( 5.
Publish Post & Index State Updates )--
```

1. Publisher drafts an article in the React/Tauri UI and selects access level (e.g. "Subscriber Only").
2. The UI passes the raw text to the local background Delegate via IPC/WebSocket.
3. The Delegate checks for an active K_{epoch} for the current month. If none exists, it generates a new one.
4. The Delegate encrypts the payload, packages attachments, and signs the updated `ContentIndexContract`.
5. The Delegate broadcasts the updated contract states to the Freenet P2P network.

5.2 Workflow B: Subscription Purchase & Automatic Key Delivery Flow

```
[ Reader Delegate ] --( 1. Fetch Profile & Tier Info )--> [ Freenet Contract ] | 2. Request LN Invoice
via NWC v [ Publisher Delegate ] --( 3. Send Lightning Invoice )--> [ Reader Delegate ] | | |
<----- 4. Execute Payment -----| | 5. Verify Invoice Preimage 6. Compute ECDH
Secret (S_i) 7. Encrypt K_epoch -> E_key,i | v [ Freenet Network ] <--( 8. Append Key Bundle to
Registry )-- | v [ Reader Delegate ] --( 9. Fetch Key Bundle, Decrypt K_epoch, Decrypt Post )--> [
Reader UI ]
```

1. **Discovery:** Reader's UI fetches the `PublisherProfileContract` and displays subscription options.
2. **Invoice Request:** Reader clicks "Subscribe". Reader's Delegate sends an encrypted peer message over Freenet to the Publisher's Delegate containing `PK_sub` and requested tier.
3. **Invoice Generation:** Publisher's Delegate generates a Lightning Network invoice with an embedded unique sub-id.
4. **Payment Execution:** Reader's Delegate executes the payment via Nostr Wallet Connect (NWC) or local WebLN wallet.
5. **Verification & Key Release:** Upon detecting payment settlement (invoice preimage receipt), Publisher's Delegate automatically derives S_i via ECDH, encrypts K_{epoch} , and appends the key bundle to the reader's record on Freenet.
6. **Decryption:** Reader's Delegate detects the new key bundle on Freenet, decrypts K_{epoch} using SK_{sub} , and unlocks all current epoch articles in the reader's local app.

6. Payment Integration & Monetization Mechanics

6.1 Nostr Wallet Connect (NWC) Integration

To make payments seamless without hosting centralized payment processors (like Stripe), Aetheria natively integrates **Nostr Wallet Connect (NIP-47)**. This allows users to link any Lightning-enabled wallet (Alby, Mutiny, Phoenix, Umbrel) to their local Delegate.

```
// Example NWC Invoice Request Payload from Reader Delegate
{
  "method": "make_invoice",
  "params": {
    "amount_msat": 5000000, // 5,000 Satoshis
    "description": "Aetheria Subscription: 2026-08 (Subscriber: pk_sub_xyz)",
    "expiry": 3600
  }
}
```

6.2 Subscription Renewal & Expiration Logic

Subscription State	Trigger Event	Delegate Action
Active	Payment verified for current Epoch E .	Publisher Delegate provides E_{key} bundle for Epoch E .
Pending Renewal	Epoch transitions from E to $E+1$.	Reader Delegate prompts auto-payment via NWC micro-rail.
Expired	Renewal payment fails or canceled.	Publisher Delegate omits Reader's PK_{sub} during Epoch $E+1$ key distribution. Reader retains access to past paid epochs (E) but cannot decrypt new posts.

6.3 Creator Platform Monetization (Protocol & Client Fee Split)

Because the protocol is serverless, platform maintenance can be monetized via open-source client application fees:

- **Direct Peer Payments:** 100% of subscriber payments go directly from reader wallet to publisher wallet by default.
- **Optional App Split:** Client UI builds can optionally configure a 1-2% split on invoice generation routed to developer node infrastructure for optional push notifications or relay indexing services.

7. Security Architecture, Threat Modeling & Mitigations

Threat / Attack Vector	Impact	Protocol Mitigation Strategy
Sybil Spam Attacks	Malicious actors flood Freenet contracts with fake subscription requests or garbage data.	Freenet WASM state contracts enforce proof-of-work puzzle validation or require attached valid micro-payment preimages before state state mutations are accepted.
Epoch Key Leakage	A malicious paid subscriber leaks K_{epoch} publicly.	Forward secrecy is preserved across epochs (leaked key only decrypts current month's articles). Future epochs use newly generated keys. Watermarking/fingerprinting encrypted payload chunks can identify the leaking subscriber.

Threat / Attack Vector	Impact	Protocol Mitigation Strategy
Publisher Offline / Host Downtime	Publisher's desktop is offline when a reader pays, delaying key delivery.	Delegates can run on low-power background nodes (Raspberry Pi / VPS) or publish pre-encrypted key bundles for authorized time-locked epoch vouchers.
Traffic Analysis & ISP Censorship	ISPs monitor network traffic to identify who is reading specific publications.	Freenet routes packets through multi-hop P2P obfuscated UDP tunnels, preventing external network observers from linking IP addresses to specific contract requests.
Network Eviction (Rarely Accessed Data)	Old articles fade from Freenet due to low request frequency.	Publisher and subscriber Delegates run automated "Pinning Daemon" threads that periodically fetch and re-seed historical publication contracts to ensure permanence.

8. Developer Hand-off & Implementation Roadmap

8.1 Recommended Technology Stack

- **WASM Contracts:** Rust (``freenet-stdlib``, ``freenet-abi``) compiled to ``wasm32-unknown-unknown``.
- **Local Delegate Daemon:** Rust daemon utilizing Tokio async runtime, SQLite for local indexing, and WebSockets for UI communication.
- **Frontend Client:** React 18, TypeScript, TailwindCSS, bundled inside Tauri (Desktop) or running as a local single-page web application.
- **Cryptographic Libraries:** ``ed25519-dalek`` (Signatures), ``k256`` (Secp256k1/ECDH), ``aes-gcm`` (Symmetric Encryption).

8.2 Milestone Execution Phases

Phase 1: WASM State Contracts & Core Engine (Weeks 1–4)

- Implement ``PublisherProfile``, ``ContentIndex``, and ``PostData`` Rust WASM contracts.
- Write unit test suites using the Freenet contract simulator.
- Verify CRDT state merging logic for append-only content logs.

Phase 2: Local Delegate & Cryptographic Pipeline (Weeks 5–8)

- Build local Rust Delegate daemon with local SQLite storage.
- Implement key generator (``Ed25519`` master keys & ``Secp256k1`` subscriber keys).
- Build automated AES-256-GCM encryption/decryption pipeline for posts and media.
- Implement ECDH key exchange flow between Publisher and Subscriber Delegates.

Phase 3: NWC Payment Engine & Auto-Key Delivery (Weeks 9–11)

- Integrate Nostr Wallet Connect (NIP-47) RPC client into Delegate daemon.
- Build automated preimage verification listener to trigger key bundle generation.
- Implement epoch transition schedule manager.

Phase 4: Frontend UI, Pinning Daemon & Launch (Weeks 12–16)

- Build modern React/Tailwind UI (Rich text editor, subscriber manager, reading feed).
- Integrate Tauri wrapper for native desktop app compilation (macOS, Windows, Linux).
- Implement automated background contract pinning engine.
- Conduct end-to-end security audit and deployment testnet release.